

CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

AI FAQ Assistant API is a robust RESTful backend application designed to help users create, manage, search and retrieve Frequently Asked Questions while using Generative AI to provide intelligent answers to user queries.

The system provides a secure and structured API platform with **User Authentication, FAQ Management, FAQ Search, AI Answer Generation and AI FAQ Generation** features. Users can register and log in securely using JWT authentication, while authenticated users can create, update and delete their own FAQ records.

The platform contains an **AI Answer Generator**, which accepts arbitrary user questions and generates concise and meaningful answers using Google's Gemini Generative AI model. The **AI FAQ Generator** accepts a topic and automatically generates a structured FAQ question-and-answer pair. The project is developed using **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt and the official Google Gemini SDK (@google/genai)**. The application follows the **MVC (Model-View-Controller) architecture** to provide a modular, maintainable and scalable backend structure.

Main Features

- User Registration
- Secure User Login
- JWT Authentication
- User Profile Management
- Password Hashing using bcrypt
- FAQ Creation
- FAQ Retrieval
- FAQ Updating
- FAQ Deletion
- FAQ Search
- Category Validation
- AI Answer Generation
- AI FAQ Generation
- Structured JSON AI Responses
- Centralized Error Handling
- MongoDB Database Integration
- Postman API Testing

Scenarios

Scenario 1: User Authentication

A new user registers with their name, email and password. The system securely hashes the password using bcrypt and stores the user information in MongoDB. The user can then log in using their registered email and password. After successful authentication, the system generates a JWT token that can be used to access protected APIs.

Scenario 2: FAQ Management

An authenticated user creates an FAQ by providing a question, answer and category such as **Technology, Education, Health, Banking or General**. The FAQ is stored in MongoDB. The creator can later update or delete the FAQ, while other users can access public FAQ information.

Scenario 3: AI Answer Generation

A user submits a question such as **“What is Artificial Intelligence?”** through the AI Answer Generator endpoint. The backend sends the request to Google Gemini and returns an AI-generated answer in a structured JSON response.

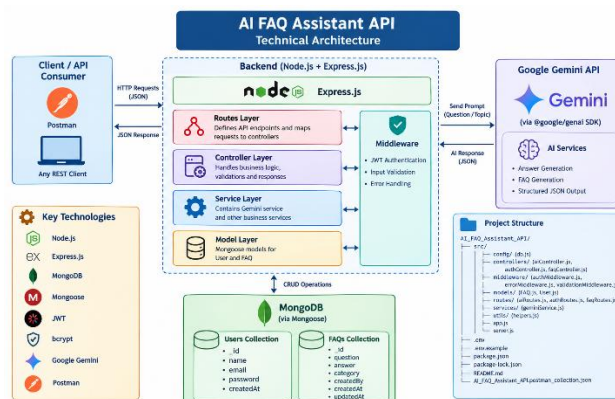
Scenario 4: AI FAQ Generation

A user provides a topic such as **“MongoDB”**. The AI FAQ Generator communicates with Gemini and automatically creates a structured FAQ containing a relevant question and answer. The generated FAQ can then be stored using the FAQ creation endpoint.

Technical Architecture:

The **AI FAQ Assistant API** uses a modular MVC-based backend architecture. The client communicates with the Express.js REST API, which handles authentication, validation, FAQ operations and AI requests.

The backend communicates with **MongoDB through Mongoose** for storing users and FAQs. For AI-powered functionality, the backend communicates with **Google Gemini through the official @google/genai SDK**.



Architecture Components

1. Client Layer

Postman or any REST API client sends HTTP requests to the backend.

2. Route Layer

Routes define API endpoints for authentication, FAQ management and AI services.

3. Controller Layer

Controllers process requests and coordinate the required business logic.

4. Service Layer

The Gemini service manages communication between the backend and Google Gemini.

5. Model Layer

Mongoose models define the structure and validation rules for users and FAQs.

6. Database Layer

MongoDB stores user and FAQ information.

7. Middleware Layer

Middleware handles JWT authentication, request validation and centralized error handling.

Pre-requisites:

- **Node.js Knowledge:** Basic knowledge of Node.js and npm.
- **Express.js Knowledge:** Understanding of REST APIs and Express.js routing.
- **MongoDB Knowledge:** Basic understanding of MongoDB databases and collections.
- **Mongoose Knowledge:** Understanding of MongoDB schema and model operations.
- **JavaScript Programming:** Basic JavaScript and asynchronous programming knowledge.
- **JWT Authentication:** Understanding of token-based authentication.
- **Google Gemini API Familiarity:** Basic understanding of Generative AI API integration.
- **Postman:** Knowledge of REST API testing using Postman.
- **Visual Studio Code:** Recommended development environment.
- **Environment Configuration:** Understanding of .env files and API key management.

Project Workflow:

Milestone 1: Project Architecture and Technology Selection

- **Activity 1.1:** Set up Node.js development environment.
- **Activity 1.2:** Select Express.js for REST API development.
- **Activity 1.3:** Configure MongoDB and Mongoose.
- **Activity 1.4:** Define MVC architecture and project structure.
- **Activity 1.5:** Configure Google Gemini API integration.

Milestone 2: User Authentication Development

- **Activity 2.1:** Develop user registration.
- **Activity 2.2:** Implement password hashing using bcrypt.
- **Activity 2.3:** Develop login functionality.
- **Activity 2.4:** Generate JWT authentication tokens.
- **Activity 2.5:** Develop protected user profile endpoint.

Milestone 3: FAQ Management Development

- **Activity 3.1:** Create FAQ Mongoose schema.
- **Activity 3.2:** Implement FAQ creation.
- **Activity 3.3:** Implement FAQ retrieval.
- **Activity 3.4:** Implement FAQ update and deletion.
- **Activity 3.5:** Implement FAQ search.

Milestone 4: AI Integration

- **Activity 4.1:** Configure Gemini API key.
- **Activity 4.2:** Develop AI Answer Generator.
- **Activity 4.3:** Develop AI FAQ Generator.
- **Activity 4.4:** Implement structured JSON responses.
- **Activity 4.5:** Validate AI-generated output.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test APIs using Postman.
- **Activity 5.2:** Test authentication and protected routes.
- **Activity 5.3:** Test FAQ CRUD operations.
- **Activity 5.4:** Test Gemini AI endpoints.
- **Activity 5.5:** Verify error handling and API performance.

Milestone 6: Conclusion

Evaluate the completed API, document the system and identify future improvements.

Milestone 1: Project Architecture and Technology Selection

This milestone focuses on designing the architecture of the AI FAQ Assistant API and selecting technologies required for implementation.

The application uses **Node.js and Express.js** for backend development, **MongoDB with Mongoose** for data storage, **JWT and bcrypt** for authentication and **Google Gemini** for Generative AI functionality.

Activity 1.1: Set Up Node.js Environment

Install Node.js and npm on the development system.

Verify the installation:

```
node --version
```

```
npm --version
```

Create the project directory and initialize the Node.js project:

```
mkdir AI_FAQ_Assistant_API
```

```
cd AI_FAQ_Assistant_API
```

```
npm init -y
```

Activity 1.2: Configure MongoDB

MongoDB is used as the database for storing users and FAQs.

The application can use a local MongoDB server or MongoDB Atlas.

Example local connection:

```
mongodb://127.0.0.1:27017/ai_faq_assistant
```

The MongoDB connection is configured using the environment variable:

```
MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant
```

The db.js configuration file establishes the connection between the Express application and MongoDB.

Activity 1.3: Configure JWT and Security

JWT is used to protect private APIs.

The JWT secret is stored securely in the .env file:

```
JWT_SECRET=your_secure_jwt_secret
```

Passwords are not stored as plain text. The application uses **bcrypt** to hash passwords before storing them in MongoDB.

The authentication middleware verifies the JWT token before allowing access to protected endpoints.

ACTIVITY 1.4

Define the Architecture of the Application

The application follows the **MVC architecture**.

Model

The model layer contains:

User.js

FAQ.js

These files define MongoDB schemas using Mongoose.

View

Since the project is a RESTful backend API, Postman or another API client acts as the client interface for testing and consuming API responses.

Controller

Controllers contain the main application logic:

authController.js

faqController.js

aiController.js

Routes

Routes connect HTTP requests to controllers:

authRoutes.js

faqRoutes.js

aiRoutes.js

Services

The Gemini service manages communication with Google's Gemini API.

geminiService.js

Middleware

Middleware provides:

- JWT authentication
- Input validation
- Centralized error handling

Milestone 2: Core Functionalities Development

The authentication module provides secure registration, login and profile management.

Activity 2.1: User Registration

The registration endpoint is:

POST /api/auth/register

Request:

```
{  
  
  "name": "John Doe",  
  
  "email": "john@gmail.com",  
  
  "password": "123456"
```

```
}
```

The system validates the input, checks whether the email already exists and securely hashes the password using bcrypt before storing the user.

Activity 2.2: User Login

The login endpoint is:

POST /api/auth/login

Request:

```
{
```

```
  "email": "john@gmail.com",
```

```
  "password": "123456"
```

```
}
```

After successful authentication, the API returns a JWT token.

Milestone 3: App.py Development

This milestone focuses on creating and managing FAQ records.

Activity 3.1: FAQ Creation

The endpoint is:

POST /api/faqs

Request:

```
{
```

```
  "question": "What is Node.js?",
```

```
"answer": "Node.js is a JavaScript runtime environment.",  
  
"category": "Technology"  
  
}
```

Valid categories are:

Technology

Education

Health

Banking

General

Activity 3.2: Retrieve FAQs

Get all FAQs:

GET /api/faqs

Get a specific FAQ:

GET /api/faqs/:id

These endpoints allow users to retrieve FAQ information.

Activity 3.3: Update FAQ

The endpoint is:

PUT /api/faqs/:id

Only the creator of the FAQ can update it.

Activity 3.4: Delete FAQ

The endpoint is:

DELETE /api/faqs/:id

The system verifies ownership before deleting the FAQ.

Milestone 4: Frontend Development

This milestone focuses on integrating Google Gemini into the backend.

Activity 4.1: Configure Gemini API

The Google Gemini API key is stored securely in the environment file:

GEMINI_API_KEY=your_google_gemini_api_key

The official Google Generative AI SDK is used:

@google/genai

The Gemini service manages communication between the backend and the AI model.

PAGE 12 — AI ANSWER GENERATOR

Activity 4.2: AI Answer Generator

The AI Answer Generator endpoint is:

POST /api/ai/answer

Request:

```
{  
  "question": "What is Artificial Intelligence?"  
}
```

The backend sends the question to Gemini and receives an AI-generated answer.

Example response:

```
{  
  "success": true,  
  "answer": "Artificial Intelligence is a field of computer science..."  
}
```

The feature allows users to obtain quick answers to general questions using Generative AI.

Activity 4.3: AI FAQ Generator

The endpoint is:

POST /api/ai/generate-faq

Request:

```
{
```

```
"topic": "MongoDB"
}
The AI generates:
{
  "success": true,
  "question": "What is MongoDB?",
  "answer": "MongoDB is a NoSQL document-oriented database..."
}
```

The generated question-answer pair can subsequently be stored using:
POST /api/faqs

STRUCTURED AI RESPONSE

Activity 4.4: Structured JSON Output

The AI FAQ Generator uses structured JSON output to maintain a consistent response format.

The expected structure contains:

question

answer

This makes the AI response easier for the backend to process and allows the generated FAQ to be stored directly in the database.

Benefits

- Consistent API responses
- Easier frontend integration
- Reduced response-parsing errors
- Better data validation
- Easier database storage

Activity 4.5: AI Response Validation

The application validates the AI response before returning it to the client.

If the Gemini API fails or returns an unexpected response, centralized error handling returns an appropriate error message instead of allowing the server to crash.

MILESTONE 5

Milestone 5: Testing and Deployment

The application is first tested locally before deployment.

Testing is performed using **Postman**.

The main testing areas include:

- User Registration
- User Login
- User Profile
- FAQ Creation
- FAQ Retrieval
- FAQ Update
- FAQ Delete
- FAQ Search
- AI Answer Generation
- AI FAQ Generation

LOCAL DEPLOYMENT

Activity 5.1: Preparing the Application for Local Deployment

Open the project in Visual Studio Code.

Install dependencies:

npm install

Configure the .env file:

PORT=5000

MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant

JWT_SECRET=your_secure_jwt_secret

GEMINI_API_KEY=your_google_gemini_api_key

Start MongoDB before starting the application.

LOCAL TESTING

Activity 5.2: Local Testing and Verification

Start the application using:

npm run dev

or:

npm start

The server runs on:

http://localhost:5000

Use Postman to test the API endpoints.

First register a user and then log in to obtain the JWT token.

The token is used for protected endpoints.

Test:

GET /api/auth/profile

POST /api/faqs

PUT /api/faqs/:id

DELETE /api/faqs/:id

POST /api/ai/answer

POST /api/ai/generate-faq

Verify that successful requests return appropriate JSON responses.

POSTMAN API TESTING

Activity 5.3: API Testing Using Postman

A pre-configured Postman collection is included:

AI_FAQ_Assistant_API.postman_collection.json

Authentication Testing

POST /api/auth/register

POST /api/auth/login

GET /api/auth/profile

FAQ Testing

POST /api/faqs

GET /api/faqs

GET /api/faqs/:id

PUT /api/faqs/:id

DELETE /api/faqs/:id

GET /api/faqs/search?q=MongoDB

AI Testing

POST /api/ai/answer

POST /api/ai/generate-faq

The login request can automatically save the JWT token in the Postman collection variable for subsequent protected requests.

TESTING RESULTS

Activity 5.4: Testing and Verification

The APIs are tested for:

- Functional correctness
- Authentication
- Authorization
- Input validation
- Database operations
- AI response generation
- Error handling
- Response time

Expected Testing Result

The system should:

1. Successfully register new users.
2. Prevent duplicate email registration.
3. Authenticate valid users.
4. Reject invalid login credentials.
5. Generate JWT tokens.
6. Protect private APIs.
7. Create and retrieve FAQs.
8. Allow only owners to update or delete FAQs.
9. Search FAQs using keywords.
10. Generate AI answers successfully.
11. Generate structured AI FAQ pairs.
12. Return appropriate errors for invalid requests.

API MODULES

Exploring the API Modules

Authentication Module

The authentication module manages:

- Registration
- Login
- Profile retrieval
- Password hashing
- JWT token generation
- JWT verification

FAQ Management Module

The FAQ module manages:

- Create FAQ
- Read FAQ
- Update FAQ
- Delete FAQ
- Search FAQ

AI Module

The AI module provides:

- AI Answer Generator
- AI FAQ Generator
- Structured AI responses

API REQUEST / RESPONSE SECTION

Authentication API

Register

POST /api/auth/register

Login

POST /api/auth/login

Profile

GET /api/auth/profile

FAQ API

Create

POST /api/faqs

Get All

GET /api/faqs

Get by ID

GET /api/faqs/:id

Update

PUT /api/faqs/:id

Delete

DELETE /api/faqs/:id

Search

GET /api/faqs/search?q=query

AI API

AI Answer

POST /api/ai/answer

Generate FAQ

POST /api/ai/generate-faq

For the screenshots in this section, insert your **Postman screenshots** showing the request, response, status code and response time.

CONCLUSION

Conclusion

The **AI FAQ Assistant API** is a Generative AI-powered RESTful backend application designed to simplify FAQ management and provide intelligent answers to user questions.

The project combines **Node.js, Express.js, MongoDB, Mongoose, JWT authentication, bcrypt and Google Gemini AI** to create a secure and scalable backend system. The MVC architecture separates the application into models, controllers, routes, services and middleware, making the system easier to maintain and extend.

The authentication module provides secure user registration and login using password hashing and JWT-based authorization. The FAQ management module enables authenticated users to create, update and delete their own FAQs while allowing public retrieval and search.

The integration of Google Gemini adds intelligent capabilities to the system. The **AI Answer Generator** provides answers to arbitrary questions, while the **AI FAQ Generator** automatically creates structured question-and-answer pairs from a given topic.

The project was tested using **Postman**, covering authentication, FAQ CRUD operations, search functionality and AI services. Centralized error handling and input validation improve the reliability of the application.

Overall, the **AI FAQ Assistant API** demonstrates how RESTful backend technologies and Generative AI can be combined to build a practical, secure and intelligent FAQ management platform.

Future Enhancements

The project can be further enhanced by:

- Adding a web-based frontend.
- Adding an admin dashboard.
- Implementing FAQ analytics.
- Adding multilingual AI responses.
- Adding conversation history.
- Adding role-based access control.
- Implementing FAQ recommendation using AI.
- Adding caching for frequently requested FAQs.
- Deploying the API to cloud platforms such as Render, AWS or Azure.
- Adding automated unit and integration testing.
- Implementing API rate limiting.
- Adding Swagger/OpenAPI documentation.

Project Structure to Use in the Documentation

For the **File Structure / Architecture pages**, use your actual project structure:

AI_FAQ_Assistant_API/

```
|
|
├── src/
|   ├── config/
|   │   └── db.js
|   ├── controllers/
|   │   ├── aiController.js
|   │   ├── authController.js
|   │   └── faqController.js
|   ├── middleware/
|   │   ├── authMiddleware.js
|   │   └── errorMiddleware.js
```

```
├── validationMiddleware.js
├── models/
│   ├── FAQ.js
│   └── User.js
├── routes/
│   ├── aiRoutes.js
│   ├── authRoutes.js
│   └── faqRoutes.js
├── services/
│   └── geminiService.js
├── utils/
│   └── helpers.js
├── app.js
├── server.js
├── .env
├── .env.example
├── package.json
├── package-lock.json
├── README.md
└── AI_FAQ_Assistant_API.postman_collection.json
```

One important point: unlike the sample project, your project is a **REST API backend**, so don't copy the sample's Flask, HTML frontend, Groq, LLaMA, Instagram/LinkedIn or Ngrok content. The content above replaces those parts with your actual **Node.js + Express + MongoDB + Gemini + JWT + Postman** project details.